

1. Relational Data Model

Learning Objectives

- Understand How Data is Organized in the Relational Data Model
- Understand the Different Types of Keys
- Design a Relational Database Schema for an Enterprise
- Understand the Relational Algebra Operations

1.1. Relational Data Model

The relational data model organizes the database as a set of relations and relationship among the relations. A relation is a table of values with rows and columns. In relational database terminology a table is termed as a relation, a row is termed as a tuple and a column is termed as an attribute. Any relation must have an atomic attribute or combination of two or more attributes that will uniquely identify a tuple in a relation. It is termed as candidate key. If there is only one candidate key by default it becomes the primary key. If there are two or more candidate keys the designer must select the most appropriate candidate key and assign it as primary key. Others are alternate keys. A Relational Database schema for a University Database is presented with four Relations Student, Branch, Course and Enrolls.

Consider the relation Student (Table 1.1) with attributes Application Number, Roll Number, Name, Date of Birth, Gender and Branch Code.

Table 1.1: Relation STUDENT

STUDENT

APPNO	<u>ROLLNO</u>	NAME	DOB	GENDER	BCODE
2020501	202011101	ABRAHAM	24-MAR-2001	M	101
2020612	202011102	PREETHI	12-MAR-2001	F	101
2020644	202011103	MANJU	11-JAN-1999	F	101
2020412	202012101	PETER	14-FEB-2001	M	103
2020555	202012102	LINI	11-MAR-2001	F	103
2020443	202015101	JAMES	10-FEB-2001	M	102
2020444	202015102	CAROLINE	13-MAR-2001	F	102

The STUDENT relation has two candidate keys. One candidate key is APPNO and the other candidate key is ROLLNO. Among these two candidate keys, the candidate key ROLLNO is assigned as primary key.

Consider the relation Branch (Table 1.2) with attributes Branch Code and Branch Name.

Table 1.2: Relation BRANCH

BRANCH

<u>BCODE</u>	BNAME
101	COMPUTER SCIENCE
102	INFORMATION SCIENCE
103	CIVIL
104	MECHANICAL
105	MINING

The BRANCH relation has only one candidate key BCODE. Hence, the primary key of relation BRANCH is attribute BCODE.

Consider the relation Course (Table 1.3) with attributes Course Code, Course Name and Credits.

Table 1.3: Relation COURSE

COURSE

<u>CCODE</u>	CNAME	CREDITS
CA133	DATABASE MANagementsYSTEMS	4
CA134	COMPUTER ORGANIZATION	3
CA135	C PROGRAMMING	3
CA136	C++ PROGRAMMING	3
CA137	R PROGRAMMING	4

The COURSE relation has only one candidate key CCODE. Hence, the primary key of relation COURSE is attribute CCODE.

Consider the relation Enrolls (Table 1.4) with Attributes Roll Number, Course Code, Session and Grade. A Student has to enroll for a set of Courses prescribed in the Curriculum during a Semester (Session). If a Student is Unsuccessful (RA – Re Appearance) for a Course, he / she should enroll for the next Session. A Student has successfully completed the Course if he / she secure a Grade S / A / B / C / D / E.

Table 2.4: Relation ENROLLS

ENROLLS

<u>ROLLNO</u>	<u>CCODE</u>	<u>SESS</u>	GRADE
202011101	CA133	NOV2022	A
202011101	CA134	NOV2022	A
202011101	CA135	NOV2022	RA
202011102	CA133	NOV2022	B+
202011102	CA134	NOV2022	A
202011102	CA135	NOV2022	RA
202011101	CA135	APR2023	
202011102	CA135	APR2023	

The relation ENROLLS has only one candidate key (ROLLNO, CCODE, SESS). The candidate key is formed by three attributes (composite key). Hence the primary key of relation ENROLS is combination of attributes (ROLLNO, CCODE, SESS).

10.1.1. Properties of Candidate Key

- Unique – Duplicate values are not allowed
- Not Null – Null values are not allowed
- Minimal –Removal of some attribute from the candidate key means the uniqueness property will no more hold (The question of minimal property comes in to picture when the candidate key is composite).

✚ In relation ENROLLS the candidate key is (ROLLNO, CCODE, SESS).

✓ Case 1: (~~ROLLNO~~, CCODE, SESS)

If attribute ROLLNO is removed from the candidate key (CCODE, SESS) will not uniquely identify a tuple.

✓ Case 2: (ROLLNO, ~~CCODE~~, SESS)

If attribute CCODE is removed from the candidate key (ROLLNO, SESS) will not uniquely identify a tuple.

✓ Case 3: (ROLLNO, CCODE, ~~SESS~~)

If attribute SESS from the candidate key (ROLLNO, CCODE) will not uniquely identify a tuple.

10.1.2. Super Key

A super key is a unique key; however, a super key will contain extraneous attributes which means there exists some attribute(s) a_i belonging to the super key which when removed the uniqueness property still holds.

Consider the relation STUDENT; the attributes (ROLLNO, NAME, DOB) taken together will uniquely identify a student. However, if attributes NAME and DOB are removed the attribute ROLLNO will uniquely identify a student. A candidate key is a minimal super key.

1.1.3. Foreign Key

An attribute 'a' of relation R is said to be a foreign key if it references attribute 'b' of the same relation R or another relation S where b is a primary key and this constraint is termed as referential integrity constraint.

The relational database schema with referential integrity constraints for the University database with relations Student, Branch, Course and Enrolls is presented below (Figure 10.1). The primary key of each relation is underlined.

STUDENT

APPNO	<u>ROLLNO</u>	NAME	DOB	GENDER	BCODE
-------	---------------	------	-----	--------	-------

BRANCH

<u>BCODE</u>	BNAME
--------------	-------

COURSE

<u>CCODE</u>	CNAME	CREDITS
--------------	-------	---------

ENROLLS

<u>ROLLNO</u>	<u>CCODE</u>	<u>SESS</u>	GRADE
---------------	--------------	-------------	-------

Figure 1.1: University Database Schema with Referential Integrity Constraints

- Attribute BCODE in Student relation is a foreign key referencing attribute BCODE of Branch relation.

- Attribute ROLLNO in Enrolls relation is a foreign key referencing attribute ROLLNO of Student relation.
- Attribute CCODE in Enrolls relation is a foreign key referencing attribute CCODE of Course relation.

1.2.Relational Algebra

Relational algebra is an integral part of the relational data model and its operations can be divided into two groups. One group includes set operations namely Union, Intersection, Set Difference and Cartesian Product from mathematical set theory since each relation is defined to be a set of tuples in the relational data model. The other group consists of operations specific to the relational data model; these include Select, Project, and Join operations. There are also other operations specific to the relational data model.

- Relational algebra operations can be used to specify retrievals.
- A sequence of relational algebra operations forms a relational algebra expression.
- A relational algebra expression forms a new relation after applying algebraic operator (s) to an existing set of relations.
- The use of is for relational algebra expression is querying a database to retrieve information.
- The result of any relational algebra operation is also a relation.
- Select and Project operations are unary operations, which operate on a single relation.
- Cartesian Product and Join operations are binary operations, which operate on two relations.

The Select, Project, Cartesian Product and Join operations are outlined below.

1.2.1. Select (σ) Operation

The Select (σ) operation in relational algebra is a horizontal filter that returns subset of tuples satisfying the selection condition. The selection condition can be formed using the relational operators(= , < , > , $\leq$, $\geq$). Selection condition can have Boolean operators AND, OR and NOT.

- $\sigma_{(CONDITION)}(RELATION NAME)$
- $\sigma_{(CONDITION 1 AND CONDITION 2)}(RELATION NAME)$

Select operation is an unary operation and must always follow a condition.

- $\sigma_{(CONDITION)}(RELATION NAME) - Correct$
- $\sigma_{(BCODE=101)}(STUDENT) - Correct$
- $\sigma_{(ARRRIBUTE LIST)}(RELATION NAME) - Wrong$
- $\sigma_{(ROLLNO,NAME)}(STUDENT) - Wrong$

Example 1: List the tuples pertaining to students who belong to branch code 101 (Table 1.5).

Table 1.5: Example of Select Operation in Relational Algebra

$$R_1 \leftarrow \sigma_{(BCODE=101)}(STUDENT)$$

APPNO	<u>ROLLNO</u>	NAME	DOB	GENDER	BCODE
2020501	202011101	ABRAHAM	24-MAR-2001	M	101
2020612	202011102	PREETHI	12-MAR-2001	F	101
2020644	202011103	MANJU	11-JAN-1999	F	101

Example 2: List the tuples pertaining to girl students who belong to branch code 101 (Table 10.6).

Table 10.6: Example of Select Operation in Relational Algebra with Boolean Operator

$$R_2 \leftarrow \sigma_{(BCODE=101 \text{ AND } GENDER='F')}(STUDENT)$$

APPNO	<u>ROLLNO</u>	NAME	DOB	GENDER	BCODE
2020612	202011102	PREETHI	12-MAR-2001	F	101
2020644	202011103	MANJU	11-JAN-1999	F	101

1.2.2. Project (π) Operation

The project (π) operation in relational algebra is a vertical filter that returns subset of attributes.

- $\pi_{(Attribute List)}(RELATION NAME)$

Project operation is an unary operation and must always follow an attribute list.

- $\pi_{(ARRRIBUTE LIST)}(RELATION NAME) - Correct$
- $\pi_{(ROLLNO,NAME)}(STUDENT) - Correct$
- $\pi_{(CONDITION)}(RELATION NAME) - Wrong$
- $\pi_{(BCODE=101)}(STUDENT) - Wrong$

Example 1: List the roll number and name of all students (Table 1.7).

Table 1.7: Example of Project Operation in Relational Algebra

$$R_3 \leftarrow \pi_{(ROLLNO, NAME)}(STUDENT)$$

<u>ROLLNO</u>	NAME
202011101	ABRAHAM
202011102	PREETHI
202011103	MANJU
202012101	PETER
202012102	LINI
202015101	JAMES
202015102	MARY

A relational algebra expression can be formed using both Select and Project operations. In this case, the Selection is done first followed by Projection.

Example 2: List the roll number and name of all girl students who belong to branch code 101 (Table 1.8).

Table 1.8: Example of Relational Algebra Expression with Select and Project Operations

$$R_4 \leftarrow \pi_{(ROLLNO, NAME)}(\sigma_{(BCODE=101 \text{ AND } GENDER='F')}(STUDENT))$$

<u>ROLLNO</u>	NAME
202011102	PREETHI
202011103	MANJU

1.2.3. Cartesian Product ($\times$)

Cartesian product of two relations is similar to Cartesian product of two sets; if set A has m elements and if set B has n elements $A \times B$ will have $m \times n$ elements. When a Cartesian product is performed between two relations R and S, if relation R has m tuples and if relation S has n tuples $R \times S$ will have $m \times n$ tuples.

$$A = \{1, 2, 3\}$$

$$B = \{4, 5, 6\}$$

$$A \times B = \{(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)\}$$

In the above example set A has 3 elements; set B has 3 elements; set $A \times B$ has 9 elements.

The relation STUDENT presented in Table 1.1 has 7 tuples and the relation BRANCH presented in Table 1.2 has 5 tuples. Cartesian product of relation STUDENT and relation BRANCH will have 35 tuples.

To illustrate Cartesian Product, Equi-Join and Outer Joins consider the PLAYER relation with four tuples (Table 1.9) and the GAME relation with four tuples (Table 1.10).

Table 1.9: Relation PLAYER

PLAYER

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE
202011101	ABRAHAM	24-MAR-2001	M	201
202011102	PREETHI	12-MAR-2001	F	202
202011103	MANJU	11-JAN-1999	F	
202012101	PETER	14-FEB-2001	M	

Table 1.10: Relation GAME

GAME

<u>GCODE</u>	GNAME
201	CRICKET
202	HOCKEY
203	TABLE TENNIS
204	FOOT BALL

The relation PLAYER presented in Table 1.9 has 4 tuples and the relation GAME presented in Table 1.10 has 4 tuples. Cartesian product of relation PLAYER and relation GAME has 16 tuples (Table 1.11).

Table 1.11: Cartesian Product of Relation Player and Relation GAME**STUDENT × GAME**

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE	GCODE	GNAME
202011101	ABRAHAM	24-MAR-2001	M	201	201	CRICKET
202011101	ABRAHAM	24-MAR-2001	M	201	202	HOCKEY
202011101	ABRAHAM	24-MAR-2001	M	201	203	TABLE TENNIS
202011101	ABRAHAM	24-MAR-2001	M	201	204	FOOT BALL
202011102	PREETHI	12-MAR-2001	F	202	201	CRICKET
202011102	PREETHI	12-MAR-2001	F	202	202	HOCKEY
202011102	PREETHI	12-MAR-2001	F	202	203	TABLE TENNIS
202011102	PREETHI	12-MAR-2001	F	202	204	FOOT BALL
202011103	MANJU	11-JAN-1999	F		201	CRICKET
202011103	MANJU	11-JAN-1999	F		202	HOCKEY
202011103	MANJU	11-JAN-1999	F		203	TABLE TENNIS
202011103	MANJU	11-JAN-1999	F		204	FOOT BALL
202012101	PETER	14-FEB-2001	M		201	CRICKET
202012101	PETER	14-FEB-2001	M		202	HOCKEY
202012101	PETER	14-FEB-2001	M		203	TABLE TENNIS
202012101	PETER	14-FEB-2001	M		204	FOOT BALL

1.2.4. Equi-Join

When an equi-join is done between two relations R and S, the join attribute of each tuple in R will be compared with the join attribute of each tuple in S. If the join attribute of a tuple in R matches with the join attribute of a tuple in S both the tuples will be joined. If the join attribute of a tuple in R does not match with any of the join attribute of the tuples in S or if the join attribute of a tuple in S does not match with any of the join attribute of the tuples in R the tuples will be discarded.

Equi-Join of relation PLAYER and relation GAME has 2 tuples (Table 1.12). It can be inferred from Table 1.12 two tuples from the PLAYER relation and two tuples from the GAME relation are not there in the result of equi-join. If all the tuples in the PLAYER relation and all the tuples in the GAME relation have to be included a type of Join called Outer Join must be used.

Table 1.12: Equi-Join of Relation Player and Relation GAME

PLAYER ⋈ GAME

GCODE = GCODE

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE	GCODE	GNAME
202011101	ABRAHAM	24-MAR-2001	M	201	201	CRICKET
202011102	PREETHI	12-MAR-2001	F	202	202	HOCKEY

1.2.5. Outer Join

When an equi-join is done between two relations R and S, if the join attribute of a tuple in R does not match with any of the join attribute of the tuples in S or if the join attribute of a tuple in S does not match with any of the join attribute of the tuples in R the tuples will be discarded. If all the tuples in the relation R and all the tuples in the relation S have to be included an Outer Join must be used. The types of Outer Join are listed below.

- Left Outer Join
- Right Outer Join
- Full Outer Join

1.2.5.1. Left Outer Join

When a left outer join is done between two relations R and S, if the join attribute of a tuple in R does not match with the join attribute of any of the tuple in S the tuple in R will be retained with null padded for the attributes in S. Left Outer Join of relation PLAYER and relation GAME (Table 1.13).

Table 1.13: Left Outer Join of Relation Player and Relation GAME

PLAYER ⋈ GAME

GCODE = GCODE

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE	GCODE	GNAME
202011101	ABRAHAM	24-MAR-2001	M	201	201	CRICKET
202011102	PREETHI	12-MAR-2001	F	202	202	HOCKEY
202011103	MANJU	11-JAN-1999	F			
202012101	PETER	14-FEB-2001	M			

1.2.5.2. Right Outer Join

When a right outer join is done between two relations R and S, if the join attribute of a tuple in S does not match with the join attribute of any of the tuple in R the tuple in R will be retained with null padded for the attributes in S. Right Outer Join of relation PLAYER and relation GAME (Table 1.14).

Table 1.14: Right Outer Join of Relation Player and Relation GAME

PLAYER $\bowtie$ GAME

GCODE = GCODE

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE	GCODE	GNAME
202011101	ABRAHAM	24-MAR-2001	M	201	201	CRICKET
202011102	PREETHI	12-MAR-2001	F	202	202	HOCKEY
					203	TABLE TENNIS
					204	FOOT BALL

1.2.5.3. Full Outer Join

Full Outer Join is a union of Left Outer Join and Right Outer Join. Full Outer Join of relation PLAYER and relation GAME (Table 1.15).

Table 1.15: Full Outer Join of Relation Player and Relation GAME

PLAYER $\bowtie$ GAME

GCODE = GCODE

<u>ROLLNO</u>	NAME	DOB	GENDER	GCODE	GCODE	GNAME
202011101	ABRAHAM	24-MAR-2001	M	201	201	CRICKET
202011102	PREETHI	12-MAR-2001	F	202	202	HOCKEY
202011103	MANJU	11-JAN-1999	F			
202012101	PETER	14-FEB-2001	M			
					203	TABLE TENNIS
					204	FOOT BALL

Learning Outcomes

Learner would have gained knowledge on the following:

- Organization of Data in the Relational Data Model.
- Types of Keys.
- Relational Algebra Operations.
- Designing a Relational Database Schema for an Enterprise.

Short Questions

1. Outline a primary key with an example.
2. Outline referential integrity with an example.

Long Question

1. Outline the different types of joins in relational algebra with an example.

Multiple Choice Questions

1. Select operation in relational algebra is a _____ operator.
 - i. unary
 - ii. binary
 - iii. ternary
 - iv. n-ary

Answer: i.

2. Project operation in relational algebra is a _____ operator.
 - i. unary
 - ii. binary
 - iii. ternary
 - iv. n-ary

Answer: i.

3. Cartesian product operation in relational algebra is a _____ operator.
 - i. unary
 - ii. binary
 - iii. ternary
 - iv. n-ary

Answer: ii.

References

[1] Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Seventh Edition, Pearson Education, 2016.

2. Structured Query Language – Basics

Learning Objectives

- Understand the Need for Structured Query Language
- Gain Knowledge on Creating Relations and Enforcing Integrity Constraints Using Data Definition Language
- Understand Data Manipulation Language and Data Query Language
- Understand Data Control Language and Transaction Control Language

2.1. Structured Query Language

Structured Query Language (SQL) is a domain-specific language used to interact with a relational database management system. SQL is used to create, construct and manage relational database management systems. Relational database management systems namely MySQL, MS Access, Oracle, Sybase, Informix, PostgreSQL, IBM DB2 and Microsoft SQL Server use SQL as standard database language. Other open source and commercial relational database management systems also follow the SQL standard. Generally, SQL commands are grouped into five categories (Table 2.1).

Table 2.1: Categories of SQL Commands

Data Definition Language (DDL)	Used to create and modify the schema of the database and its objects.
Data Manipulation Language (DML)	Used to add, update and delete data.
Data Query Language (DQL)	Used to retrieve certain records from one or more relations.
Data Control Language (DCL)	Used to deal with the rights, permissions, and other controls of the database system.
Transaction Control Language (TCL)	Used with DML commands like INSERT, DELETE and UPDATE.

2.1.1. Data Definition Language (DDL)

DDL is used to create the structure of a database by enforcing integrity constraints. The integrity constraints are outlined below.

- Primary Key Constraint

- ✚ Any relation must have an atomic attribute or combination of two or more attributes that will uniquely identify a tuple in a relation. It is termed as candidate key. If there is only one candidate key by default it becomes the primary key. If there are two or more candidate keys the designer must select the most appropriate candidate key and assign it as primary key. Others are alternate keys.
- Foreign Key Constraint
 - ✚ An attribute 'a' of relation R is said to be a foreign key if it references attribute 'b' of the same relation R or another relation S where b is a primary key and this constraint is termed as referential integrity constraint.
- Unique Constraint
 - ✚ Ensures there are no duplicate values for an attribute in a relation.
- Not Null Constraint
 - ✚ Ensures there are no null values for an attribute in a relation.
- Check Constraint
 - ✚ Ensures that all values of an attribute satisfy certain conditions.
- Default Constraint
 - ✚ Assigns a default value to an attribute when the INSERT INTO statement does not assign a specific value.

DDL commands include CREATE, ALTER, RENAME, DROP and TRUNCATE.

- CREATE command is used to create a new relation, a view of a relation, or other object in a relational database (for example create an index to create and retrieve data from the database quickly).
- ALTER command is used to modify an existing database object, such as a relation.
- RENAME command used to rename a relation or an attribute in a relation.
- DROP command is used to delete an entire relation, a view of a relation or other object in the database
- TRUNCATE command is used to delete all the rows in a relation.

2.1.2. Data Manipulation Language (DML)

DML is used to add record (s) to a relation, update record (s) and delete record (s) from a relation.

- INSERT INTO statement is used to insert a record to a relation.
- UPDATE statement is used to update the attribute value (s) of one or more records in a relation.
- DELETE statement is used to delete one or more records from a relation.

2.1.3. Data Query Language (DQL)

DQL is used to retrieve certain data from one or more relations. DQL is considered to be part of DML.

- A query in SQL can consist of up to six clauses namely SELECT, FROM, WHERE, GROUP BY, HAVING and ORDER BY. However, only the SELECT clause and the FROM clause, are mandatory. The clauses are specified in the following order.

SELECT	<Attribute List>
FROM	<Relation List>
[WHERE	<Condition>]
[GROUP BY	<Grouping Attribute(s)>]
[HAVING	<Group Condition>]
[ORDER BY	<Attribute List>]

2.1.4. Data Control Language (DCL)

DCL is used to control access to data stored in a database (authorization).

- Grant command is used to provide privileges to database objects for a user.
- Revoke command is used to withdraw user privileges on database objects if any granted.

2.1.5. Transaction Control Language (TCL)

TCL are used to manage transactions within the database.

- TCL commands are COMMIT, ROLLBACK and SAVEPOINT

- The COMMIT command is used to permanently save the modifications done to the database values by the DML statements.
- The ROLLBACK command is used to undo the changes made by the DML commands. That is, the rollback command restores the database to its original state since the execution of the last COMMIT.
- COMMIT and ROLLBACK are irrevocable actions.
- The SAVEPOINT command used to temporarily save one or more transaction so that the transaction (s) can be committed or rolled back later. SAVEPOINT is useful to roll back the transaction (s) back to a certain point without rolling back the entire set of transactions.
- SAVEPOINT is a revocable action.

2.2. Data Definition Language, Data Manipulation Language, Data Query Language and Transaction Control Language

The use of CREATE TABLE, ALTER TABLE, DROP TABLE, RENAME TABLE, TRUNCATE TABLE, COMMIT, SAVEPOINT and ROLLBACK commands using ORACLE 10G Relational Database, POSTGRES SQL Relational Database and MYSQL Relational Database are outlined below.

2.2.1. ORACLE 10G Relational Database

A relation EMPLOYEE with two attributes namely ENO and ENAME is created using the following DDL command.

```
SQL> CREATE TABLE EMPLOYEE
2  (
3  ENO NUMBER(5),
4  ENAME VARCHAR(30)
5  );
```

Table created.

The datatype of attribute ENO is NUMBER (5) and the data type of attribute ENAME is VARCHAR2(30). This is an example for a poor design because no integrity constraints have been enforced.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
ENO		NUMBER(5)
ENAME		VARCHAR2(30)

INSERT INTO is an example for DML statement. Using the following INSERT INTO statements three records are inserted into the EMPLOYEE relation.

```
SQL> INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
```

```
1 row created.
```

```
SQL> INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
```

```
1 row created.
```

```
SQL> INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
```

```
1 row created.
```

SELECT statement in SQL is an example for DQL.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME
14728	JOSE
14728	MARY
	ANSU

It can be inferred that the values of attribute ENO of EMPLOYEE relation is duplicated and there is also NULL value for attribute ENO.

Using the DDL command ALTER TABLE an attempt is done to enforce PRIMARY KEY constraint on EMPLOYEE relation.

```
SQL> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO)
*
```

```
ERROR at line 1:
ORA-01449: column contains NULL values; cannot alter to NOT NULL
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there is a NULL value for the attribute ENO in EMPLOYEE relation.

Using the DML DELETE statement, the tuple that has NULL value for the attribute ENO in the EMPLOYEE relation is deleted.

```
SQL> DELETE FROM EMPLOYEE WHERE ENO IS NULL;
```

```
1 row deleted.
```

```
SQL> SELECT * FROM EMPLOYEE;
```

```
      ENO ENAME
-----
    14728 JOSE
    14728 MARY
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
SQL> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO)
*
```

```
ERROR at line 1:
ORA-02437: cannot validate (MCARSEPT2023.SYS_C0011606) - primary key violated
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there are duplicate values for the attribute ENO in EMPLOYEE relation.

Now, using the DML DELETE statement, the tuples with DUPLICATE VALUES for the attribute ENO in the EMPLOYEE relation are deleted.

```
SQL> DELETE FROM EMPLOYEE WHERE ENO=14728;
```

```
2 rows deleted.
```

Now, it can be inferred that there are no tuples in the EMPLOYEE relation.

```
SQL> SELECT * FROM EMPLOYEE;
```

```
no rows selected
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
SQL> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
```

```
Table altered.
```

It can be inferred that the PRIMARY KEY constraint is enforced on attribute ENO of EMPLOYEE relation.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
-----	-----	-----
ENO	NOT NULL	NUMBER(5)
ENAME		VARCHAR2(30)

Now, an attempt is done to insert DUPLICATE values and NULL values for the attribute ENO of EMPLOYEE relation.

```
SQL> INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
```

```
1 row created.
```

```
SQL> INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
```

```
INSERT INTO EMPLOYEE VALUES(14728, 'MARY')
```

```
*
```

```
ERROR at line 1:
```

```
ORA-00001: unique constraint (MCARSEPT2023.SYS_C0011607) violated
```

```
SQL> INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
```

```
INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU')
```

```
*
```

```
ERROR at line 1:
```

```
ORA-01400: cannot insert NULL into ("MCARSEPT2023"."EMPLOYEE"."ENO")
```

It can be inferred, since ENO is the Primary Key of EMPLOYEE relation, neither a duplicate value can be assigned to the attribute ENO nor NULL value can be assigned to attribute ENO.

```
SQL> SELECT * FROM EMPLOYEE;
```

```
      ENO ENAME
-----
14728 JOSE
```

An attribute GENDER with data type CHAR and a CHECK CONSTRAINT GENDER IN ('M','F') is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
SQL> ALTER TABLE EMPLOYEE ADD GENDER CHAR CHECK(GENDER IN('M','F'));
```

```
Table altered.
```

It can be inferred an attribute GENDER is added to the EMPLOYEE relation.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
ENO	NOT NULL	NUMBER(5)
ENAME		VARCHAR2(30)
GENDER		CHAR(1)

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G
14728	JOSE	

Now, an attempt is done to update the value of attribute GENDER of the EMPLOYEE relation using the DML UPDATE statement.

```
SQL> UPDATE EMPLOYEE SET GENDER='A' WHERE ENO=14728;
```

```
UPDATE EMPLOYEE SET GENDER='A' WHERE ENO=14728
```

```
*
```

```
ERROR at line 1:
```

```
ORA-02290: check constraint (MCARSEPT2023.SYS_C0011609) violated
```

It can be inferred that the updating the value of attribute GENDER of EMPLOYEE relation failed, since an attempt has been done to update value of GENDER with the value 'A'.

```
SQL> UPDATE EMPLOYEE SET GENDER='M' WHERE ENO=14728;
```

```
1 row updated.
```

It can be inferred that only the value 'M' / 'F' can be assigned to attribute GENDER of the EMPLOYEE relation.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G
14728	JOSE	M

A relation DEPARTMENT with two attributes namely DNO and DNAME is created using the following DDL statement. The data type of attribute DCODE is NUMBER (3) and the data type of DNAME is VARCHAR (30).

```
SQL> CREATE TABLE DEPARTMENT
2  (
3  DCODE NUMBER(3),
4  DNAME VARCHAR(32)
5  );
```

Table created.

```
SQL> DESC DEPARTMENT
```

Name	Null?	Type
DCODE		NUMBER(3)
DNAME		VARCHAR2(32)

An attribute DCODE with data type NUMBER (3) is added to the EMPLOYEE relation using the DDL command ALTER TABLE.

```
SQL> ALTER TABLE EMPLOYEE ADD DCODE NUMBER(3);
```

Table altered.

It can be inferred an attribute DCODE is added to the EMPLOYEE relation.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
ENO	NOT NULL	NUMBER(5)
ENAME		VARCHAR2(30)
GENDER		CHAR(1)
DCODE		NUMBER(3)

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
SQL> ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
      2 REFERENCES DEPARTMENT(DCODE);
REFERENCES DEPARTMENT(DCODE)
      *
```

ERROR at line 2:

ORA-02270: no matching unique or primary key for this column-list

It can be inferred that since the DEPARTMENT relation has no PRIMARY KEY Constraint, the FOREIGN KEY Constraint cannot be enforced.

```
SQL> ALTER TABLE DEPARTMENT ADD PRIMARY KEY(DCODE);
```

Table altered.

It can be inferred that primary key constraint is enforced on the DEPARTMENT relation. The PRIMARY KEY of DEPARTMENT relation is DCODE.

```
SQL> DESC DEPARTMENT
```

Name	Null?	Type
DCODE	NOT NULL	NUMBER(3)
DNAME		VARCHAR2(32)

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
SQL> ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
      2 REFERENCES DEPARTMENT(DCODE);
```

Table altered.

It can be inferred that foreign key constraint is enforced on the EMPLOYEE relation. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing the PRIMARY KEY DCODE of the DEPARTMENT relation.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE
14728	JOSE	M	

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
SQL> UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728
*
ERROR at line 1:
ORA-02291: integrity constraint (MCARSEPT2023.SYS_C0011611) violated - parent
key not found
```

It can be inferred that the UPDATE statement fails. This is because the value 101 is not present in the DEPARTMENT relation. Now, a record is inserted in the DEPARTMENT relation with the value 101 for DCODE.

```
SQL> INSERT INTO DEPARTMENT VALUES(101, 'COMPUTER SCIENCE AND ENGINEERING');

1 row created.
```

```
SQL> SELECT * FROM DEPARTMENT;
```

DCODE	DNAME
101	COMPUTER SCIENCE AND ENGINEERING

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
SQL> UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
```

1 row updated.

It can be inferred after inserting a record in the DEPARTMENT relation with the value 101 for DCODE, the value of attribute DCODE in the EMPLOYEE relation is updated.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE
14728	JOSE	M	101

An attribute PAN with data type VARCHAR (10) and an UNIQUE Constraint is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
SQL> ALTER TABLE EMPLOYEE ADD PAN VARCHAR(10) UNIQUE;
```

Table altered.

It can be inferred an attribute PAN is added to the EMPLOYEE relation.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
ENO	NOT NULL	NUMBER(5)
ENAME		VARCHAR2(30)
GENDER		CHAR(1)
DCODE		NUMBER(3)
PAN		VARCHAR2(10)

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	

Now, an attempt is done to update the value of PAN of EMPLOYEE relation pertaining to ENO: 14728 using the DML statement UPDATE.

```
SQL> UPDATE EMPLOYEE SET PAN = 'XXXXX0051R'
2 WHERE ENO = 14728;
```

1 row updated.

It can be inferred attribute PAN of EMPLOYEE relation pertaining to ENO: 14728 is updated.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command.

```
SQL> INSERT INTO EMPLOYEE VALUES
  2  (14729, 'VINI', 'F', 101, 'XXXXX0051R');
INSERT INTO EMPLOYEE VALUES
*
ERROR at line 1:
ORA-00001: unique constraint (MCARSEPT2023.SYS_C0011612) violated
```

It can be inferred that the INSERT INTO DML Command fails, since an attempt is done to insert a duplicate value for PAN.

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
SQL> INSERT INTO EMPLOYEE VALUES
  2  (14729, 'VINI', 'F', 101, NULL);

1 row created.
```

It can be inferred that the INSERT INTO DML Command succeeds, with NULL value for the value of PAN.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
SQL> ALTER TABLE EMPLOYEE MODIFY PAN NOT NULL;
ALTER TABLE EMPLOYEE MODIFY PAN NOT NULL
*
ERROR at line 1:
ORA-02296: cannot enable (MCARSEPT2023.) - null values found
```

It can be inferred that the ALTER TABLE Command fails since there is a NULL value for PAN pertaining to ENO: 14729. Now, the record pertaining to ENO: 14729 is deleted using the following command.

```
SQL> DELETE FROM EMPLOYEE WHERE ENO=14729;

1 row deleted.
```

It can be inferred that the record pertaining to ENO: 14729 is deleted.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
SQL> ALTER TABLE EMPLOYEE MODIFY PAN NOT NULL;

Table altered.
```

It can be inferred that the EMPLOYEE relation is altered.

```
SQL> DESC EMPLOYEE
```

Name	Null?	Type
ENO	NOT NULL	NUMBER(5)
ENAME		VARCHAR2(30)
GENDER		CHAR(1)
DCODE		NUMBER(3)
PAN	NOT NULL	VARCHAR2(10)

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
SQL> INSERT INTO EMPLOYEE VALUES
  2  (14729, 'VINI', 'F', 101, NULL);
(14729, 'VINI', 'F', 101, NULL)
      *
```

ERROR at line 2:
ORA-01400: cannot insert NULL into ("MCARSEPT2023"."EMPLOYEE"."PAN")

It can be inferred that the INSERT INTO DML Command fails, since a NOT NULL Constraint has been enforced.

COMMIT, ROLLBACK AND SAVEPOINT

An additional record is inserted to the DEPARTMENT relation.

```
SQL> INSERT INTO DEPARTMENT VALUES(102, 'INFORMATION SCIENCE');
```

1 row created.

```
SQL> SELECT * FROM DEPARTMENT;
```

```

      DCODE DNAME
-----
      101 COMPUTER SCIENCE AND ENGINEERING
      102 INFORMATION SCIENCE
```

Now, the COMMIT command is executed.

```
SQL> COMMIT;
```

Commit complete.

Now, the ROLLBACK command is executed.

```
SQL> ROLLBACK;
```

Rollback complete.

```
SQL> SELECT * FROM DEPARTMENT;
```

DCODE	DNAME
101	COMPUTER SCIENCE AND ENGINEERING
102	INFORMATION SCIENCE

It can be inferred that COMMIT and ROLLBACK are irrevokable actions. There is no undo for COMMIT and ROLLBACK.

The EMPLOYEE relation has one record.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R

Now, three additional records are inserted into the EMPLOYEE relation.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14729, 'VINI','F',101, 'XXXXX0052R');
```

1 row created.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14730, 'ANI','F',102, 'XXXXX0053R');
```

1 row created.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14731, 'MANJU','F',102, 'XXXXX0054R');
```

1 row created.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R
14731	MANJU	F	102	XXXXX0054R

Now, the ROLLBACK command is executed.

```
SQL> ROLLBACK;
```

Rollback complete.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R

It can be inferred that after executing the ROLLBACK command the EMPLOYEE relation has only one record.

Now, a record is inserted into relation EMPLOYEE and a SAVEPOINT A is created.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14729, 'VINI', 'F', 101, 'XXXXX0052R');
```

1 row created.

```
SQL> SAVEPOINT A;
```

Savepoint created.


```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT B is created.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14730, 'ANI', 'F', 102, 'XXXXX0053R');
```

1 row created.

```
SQL> SAVEPOINT B;
```

Savepoint created.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT C is created.

```
SQL> INSERT INTO EMPLOYEE VALUES  
2 (14731, 'MANJU', 'F', 102, 'XXXXX0054R');
```

1 row created.

```
SQL> SAVEPOINT C;
```

Savepoint created.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R
14731	MANJU	F	102	XXXXX0054R

There are four records in the EMPLOYEE relation. Now the TCL command ROLLBACK to B is executed.

```
SQL> ROLLBACK TO B;
```

Rollback complete.

```
SQL> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R

It can be inferred that only three records are there in the EMPLOYEE relation after execution of ROLLBACK TO B.

RENAME TABLE, TRUNCATE TABLE and DROP TABLE

```
SQL> RENAME EMPLOYEE TO STAFF;
```

Table renamed.

```
SQL> SELECT * FROM STAFF;
```

ENO	ENAME	G	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R

It can be inferred that relation EMPLOYEE is renamed to relation STAFF.

```
SQL> TRUNCATE TABLE STAFF;
```

Table truncated.

```
SQL> SELECT * FROM STAFF;
```

no rows selected

It can be inferred that after the relation STAFF is truncated there are no records in the STAFF relation.

Now, an attempt is done to drop the DEPARTMENT relation.

```
SQL> DROP TABLE DEPARTMENT;
```

```
DROP TABLE DEPARTMENT
```

```
*
```

```
ERROR at line 1:
```

```
ORA-02449: unique/primary keys in table referenced by foreign keys
```

It can be inferred, DCODE is a foreign key of STAFF relation referencing DCODE of DEPARTMENT relation. So, the DEPARTMENT relation cannot be dropped. First the STAFF relation has to be dropped followed by DEPARTMENT relation.

```
SQL> DROP TABLE STAFF;
```

Table dropped.

```
SQL> DROP TABLE DEPARTMENT;
```

Table dropped.

It can be inferred that relation STAFF and relation DEPARTMENT are dropped.

2.2.2. POSTGRESQL Relational Database

A relation EMPLOYEE with two attributes namely ENO and ENAME is created using the following DDL command.

```
mcarsept2023=# CREATE TABLE EMPLOYEE
mcarsept2023=# (
mcarsept2023(# ENO BIGINT,
mcarsept2023(# ENAME VARCHAR(30)
mcarsept2023(# );
CREATE TABLE
```

The datatype of attribute ENO is BIGINT and the data type of attribute ENAME is VARCHAR (30). This is an example for a poor design because no integrity constraints have been enforced.

```
mcarsept2023=# \d EMPLOYEE
              Table "public.employee"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
 eno     | bigint                 |           |          |
 ename   | character varying(30) |           |          |
```

INSERT INTO is an example for DML statement. Using the following INSERT INTO statements three records are inserted into the EMPLOYEE relation.

```
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
INSERT 0 1
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
INSERT 0 1
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
INSERT 0 1
```

SELECT statement in SQL is an example for DQL.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename
-----+-----
 14728 |   JOSE
 14728 |   MARY
      |   ANSU
(3 rows)
```

It can be inferred that the values of attribute ENO of EMPLOYEE relation is duplicated and there is also NULL value for attribute ENO.

Using the DDL command ALTER TABLE an attempt is done to enforce PRIMARY KEY constraint on EMPLOYEE relation.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ERROR:  could not create unique index "employee_pkey"
DETAIL:  Key (eno)=(14728) is duplicated.
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there are duplicate values for the attribute ENO in EMPLOYEE relation.

```
mcarsept2023=# DELETE FROM EMPLOYEE WHERE ENO=14728;
DELETE 2
```

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename
-----+-----
      |   ANSU
(1 row)
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ERROR:  column "eno" of relation "employee" contains null values
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there is a NULL value for the attribute ENO in EMPLOYEE relation.

Using the DML DELETE statement, the tuple that has NULL value for the attribute ENO in the EMPLOYEE relation is deleted.

```
mcarsept2023=# DELETE FROM EMPLOYEE WHERE ENO IS NULL;
DELETE 1
```

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
 eno | ename
-----+-----
(0 rows)
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ALTER TABLE
```

It can be inferred that the PRIMARY KEY constraint is enforced on attribute ENO of EMPLOYEE relation.

```
mcarsept2023=# \d EMPLOYEE
```

```
Table "public.employee"
Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
eno    | bigint                 |           | not null |
ename  | character varying(30) |           |          |
Indexes:
    "employee_pkey" PRIMARY KEY, btree (eno)
```

Now, an attempt is done to insert DUPLICATE values and NULL values for the attribute ENO of EMPLOYEE relation.

```
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
INSERT 0 1
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
ERROR:  duplicate key value violates unique constraint "employee_pkey"
DETAIL:  Key (eno)=(14728) already exists.
mcarsept2023=# INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
ERROR:  null value in column "eno" of relation "employee" violates not-null constraint
DETAIL:  Failing row contains (null, ANSU).
```

It can be inferred, since ENO is the Primary Key of EMPLOYEE relation, neither a duplicate value can be assigned to the attribute ENO nor NULL value can be assigned to attribute ENO.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
 eno |  ename
-----+-----
14728 | JOSE
(1 row)
```

An attribute GENDER with data type CHAR and a CHECK CONSTRAINT GENDER IN ('M','F') is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD GENDER CHAR CHECK(GENDER IN('M','F'));
ALTER TABLE
```

It can be inferred an attribute GENDER is added to the EMPLOYEE relation.

```
mcarsept2023=# \d EMPLOYEE
```

Table "public.employee"				
Column	Type	Collation	Nullable	Default
eno	bigint		not null	
ename	character varying(30)			
gender	character(1)			

```
Indexes:
```

```
    "employee_pkey" PRIMARY KEY, btree (eno)
```

```
Check constraints:
```

```
    "employee_gender_check" CHECK (gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar]))
```

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
```

eno	ename	gender
14728	JOSE	

(1 row)

Now, an attempt is done to update the value of attribute GENDER of the EMPLOYEE relation using the DML UPDATE statement.

```
mcarsept2023=# UPDATE EMPLOYEE SET GENDER='A'
```

```
mcarsept2023=# WHERE ENO=14728;
```

```
ERROR:  new row for relation "employee" violates check constraint  
"employee_gender_check"
```

```
DETAIL:  Failing row contains (14728, JOSE, A, 101).
```

It can be inferred that the updating the value of attribute GENDER of EMPLOYEE relation failed, since an attempt has been done to update value of GENDER with the value 'A'.

```
mcarsept2023=# UPDATE EMPLOYEE SET GENDER='M' WHERE ENO=14728;  
UPDATE 1
```

It can be inferred that only the value 'M' / 'F' can be assigned to attribute GENDER of the EMPLOYEE relation.


```
mcarsept2023=# SELECT * FROM EMPLOYEE;
   eno |  ename | gender
-----+-----+-----
  14728 |   JOSE |      M
(1 row)
```

A relation DEPARTMENT with two attributes namely DNO and DNAME is created using the following DDL statement. The data type of attribute DCODE is SMALLINT and the data type of DNAME is VARCHAR (30).

```
mcarsept2023=# CREATE TABLE DEPARTMENT
mcarsept2023=# (
mcarsept2023(# DCODE SMALLINT,
mcarsept2023(# DNAME VARCHAR(32)
mcarsept2023(# );
```

```
mcarsept2023=# \d DEPARTMENT
               Table "public.department"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
  dcode  | smallint               |           |          |
  dname  | character varying(32) |           |          |
```

An attribute DCODE with data type SMALLINT is added to the EMPLOYEE relation using the DDL command ALTER TABLE.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD DCODE SMALLINT;
ALTER TABLE
```

It can be inferred an attribute DCODE is added to the EMPLOYEE relation.

```
mcarsept2023=# \d EMPLOYEE
```

Table "public.employee"				
Column	Type	Collation	Nullable	Default
eno	bigint		not null	
ename	character varying(30)			
gender	character(1)			
dcode	smallint			

Indexes:

"employee_pkey" PRIMARY KEY, btree (eno)

Check constraints:

"employee_gender_check" CHECK (gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar]))

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
mcarsept2023=# REFERENCES DEPARTMENT(DCODE);
ERROR:  there is no unique constraint matching given keys
for referenced table "department"
```

It can be inferred that since the DEPARTMENT relation has no PRIMARY KEY Constraint, the FOREIGN KEY Constraint cannot be enforced.

```
mcarsept2023=# ALTER TABLE DEPARTMENT ADD PRIMARY KEY(DCODE);
ALTER TABLE
```

It can be inferred that primary key constraint is enforced on the DEPARTMENT relation. The PRIMARY KEY of DEPARTMENT relation is DCODE.

```
mcarsept2023=# \d DEPARTMENT
```

Table "public.department"				
Column	Type	Collation	Nullable	Default
dcode	smallint		not null	
dname	character varying(32)			

Indexes:

"department_pkey" PRIMARY KEY, btree (dcode)

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
mcarsept2023=# REFERENCES DEPARTMENT(DCODE);
ALTER TABLE
```

It can be inferred that foreign key constraint is enforced on the EMPLOYEE relation. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing the PRIMARY KEY DCODE of the DEPARTMENT relation.

```
mcarsept2023=# \d EMPLOYEE
```

Table "public.employee"				
Column	Type	Collation	Nullable	Default
eno	bigint		not null	
ename	character varying(30)			
gender	character(1)			
dcode	smallint			

Indexes:

```
"employee_pkey" PRIMARY KEY, btree (eno)
```

Check constraints:

```
"employee_gender_check" CHECK (gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar]))
```

Foreign-key constraints:

```
"employee_dcode_fkey" FOREIGN KEY (dcode) REFERENCES department(dcode)
```

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
```

eno	ename	gender	dcode
14728	JOSE	M	

(1 row)

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
mcarsept2023=# UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
ERROR: insert or update on table "employee"
violates foreign key constraint "employee_dcode_fkey"
DETAIL: Key (dcode)=(101) is not present in table "department".
```

It can be inferred that the UPDATE statement fails. This is because the value 101 is not present in the DEPARTMENT relation. Now, a record is inserted in the DEPARTMENT relation with the value 101 for DCODE.

```
mcarsept2023=# INSERT INTO DEPARTMENT VALUES
mcarsept2023=# (101, 'COMPUTER SCIENCE AND ENGINEERING');
INSERT 0 1
```

```
mcarsept2023=# SELECT * FROM DEPARTMENT;
 dcode |          dname
-----+-----
    101 | COMPUTER SCIENCE AND ENGINEERING
(1 row)
```

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
mcarsept2023=# UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
UPDATE 1
```

It can be inferred after inserting a record in the DEPARTMENT relation with the value 101 for DCODE, the value of attribute DCODE in the EMPLOYEE relation is updated.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
 eno |  ename  | gender | dcode
-----+-----+-----+-----
 14728 | JOSE   | M      |    101
(1 row)
```

An attribute PAN with data type VARCHAR (10) and an UNIQUE Constraint is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ADD PAN VARCHAR(10) UNIQUE;
ALTER TABLE
```

It can be inferred an attribute PAN is added to the EMPLOYEE relation.

```
mcarsept2023=# \d EMPLOYEE
```

Table "public.employee"				
Column	Type	Collation	Nullable	Default
eno	bigint		not null	
ename	character varying(30)			
gender	character(1)			
dcode	smallint			
pan	character varying(10)			

Indexes:

```
"employee_pkey" PRIMARY KEY, btree (eno)
```

```
"employee_pan_key" UNIQUE CONSTRAINT, btree (pan)
```

Check constraints:

```
"employee_gender_check" CHECK (gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar]))
```

Foreign-key constraints:

```
"employee_dcode_fkey" FOREIGN KEY (dcode) REFERENCES department(dcode)
```

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
```

eno	ename	gender	dcode	pan
14728	JOSE	M	101	

(1 row)

Now, an attempt is done to update the value of PAN of EMPLOYEE relation pertaining to ENO: 14728 using the DML statement UPDATE.

```
mcarsept2023=# UPDATE EMPLOYEE SET PAN='XXXXX0051R'
mcarsept2023=# WHERE ENO=14728;
UPDATE 1
```

It can be inferred attribute PAN of EMPLOYEE relation pertaining to ENO: 14728 is updated.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 | JOSE   | M      | 101   | XXXXX0051R
(1 row)
```

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command.

```
mcarsept2023=# INSERT INTO EMPLOYEE
mcarsept2023=# VALUES(14729, 'VINI','F',101, 'XXXXX0051R');
ERROR:  duplicate key value violates unique constraint "employee_pan_key"
DETAIL:  Key (pan)=(XXXXX0051R) already exists.
```

It can be inferred that the INSERT INTO DML Command fails, since an attempt is done to insert a duplicate value for PAN.

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
mcarsept2023=# INSERT INTO EMPLOYEE
mcarsept2023=# VALUES(14729, 'VINI','F',101, NULL);
INSERT 0 1
```

It can be inferred that the INSERT INTO DML Command succeeds, with NULL value for the value of PAN.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 | JOSE   | M      | 101   | XXXXX0051R
 14729 | VINI   | F      | 101   |
(2 rows)
```

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ALTER COLUMN PAN SET NOT NULL;
ERROR: column "pan" of relation "employee" contains null values
```

It can be inferred that the ALTER TABLE Command fails since there is a NULL value for PAN pertaining to ENO: 14729. Now, the record pertaining to ENO: 14729 is deleted using the following command.

```
mcarsept2023=# DELETE FROM EMPLOYEE WHERE ENO=14729;
DELETE 1
```

It can be inferred that the record pertaining to ENO: 14729 is deleted.

```
mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |   M    |   101 | XXXXX0051R
(1 row)
```

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
mcarsept2023=# ALTER TABLE EMPLOYEE ALTER COLUMN PAN SET NOT NULL;
ALTER TABLE
```

It can be inferred that the EMPLOYEE relation is altered.

```
mcarsept2023=# \d EMPLOYEE
```

Table "public.employee"				
Column	Type	Collation	Nullable	Default
eno	bigint		not null	
ename	character varying(30)			
gender	character(1)			
dcode	smallint			
pan	character varying(10)		not null	

```
Indexes:
```

```
    "employee_pkey" PRIMARY KEY, btree (eno)
```

```
    "employee_pan_key" UNIQUE CONSTRAINT, btree (pan)
```

```
Check constraints:
```

```
    "employee_gender_check" CHECK (gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar]))
```

```
Foreign-key constraints:
```

```
    "employee_dcode_fkey" FOREIGN KEY (dcode) REFERENCES department(dcode)
```

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
mcarsept2023=# INSERT INTO EMPLOYEE VALUES
```

```
mcarsept2023=# (14729, 'VINI','F',101, NULL);
```

```
ERROR:  null value in column "pan" of relation "employee" violates not-null constraint
DETAIL:  Failing row contains (14729, VINI, F, 101, null).
```

It can be inferred that the INSERT INTO DML Command fails, since a NOT NULL Constraint has been enforced.

COMMIT, ROLLBACK AND SAVEPOINT

An additional record is inserted to the DEPARTMENT relation.

```
mcarsept2023=# BEGIN;
```

```
BEGIN
```

```
mcarsept2023=*# INSERT INTO DEPARTMENT VALUES
```

```
mcarsept2023-*# (102, 'INFORMATION SCIENCE');
```

```
INSERT 0 1
```



```
mcarsept2023=*# SELECT * FROM DEPARTMENT;
dcode |          dname
-----+-----
    101 | COMPUTER SCIENCE AND ENGINEERING
    102 | INFORMATION SCIENCE
(2 rows)
```

Now, the COMMIT command is executed.

```
mcarsept2023=*# COMMIT;
COMMIT
```

Now, the ROLLBACK command is executed.

```
mcarsept2023=# ROLLBACK;
WARNING:  there is no transaction in progress
ROLLBACK
```

```
mcarsept2023=# SELECT * FROM DEPARTMENT;
dcode |          dname
-----+-----
    101 | COMPUTER SCIENCE AND ENGINEERING
    102 | INFORMATION SCIENCE
(2 rows)
```

It can be inferred that COMMIT and ROLLBACK are irrevokable actions.

The EMPLOYEE relation has one record.

```
   eno   |  ename  | gender | dcode |   pan
-----+-----+-----+-----+-----
  14728  | JOSE    | M      | 101   | XXXXX0051R
(1 row)
```

Now, three additional records are inserted into the EMPLOYEE relation.

```

mcarsept2023=# BEGIN;
BEGIN
mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*=# (14729, 'VINI', 'F', 101, 'XXXXX0052R');
INSERT 0 1
mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*=# (14730, 'ANI', 'F', 102, 'XXXXX0053R');
INSERT 0 1
mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*=# (14731, 'MANJU', 'F', 102, 'XXXXX0054R');
INSERT 0 1

```

```

mcarsept2023=# SELECT * FROM EMPLOYEE;
   eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |    M   |   101 | XXXXX0051R
 14729 |  VINI  |    F   |   101 | XXXXX0052R
 14730 |  ANI   |    F   |   102 | XXXXX0053R
 14731 | MANJU  |    F   |   102 | XXXXX0054R
(4 rows)

```

Now, the ROLLBACK command is executed.

```

mcarsept2023=# ROLLBACK;
ROLLBACK

```

```

mcarsept2023=# SELECT * FROM EMPLOYEE;
   eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |    M   |   101 | XXXXX0051R
(1 row)

```

It can be inferred that after executing the ROLLBACK command the EMPLOYEE relation has only one record.

Now, a record is inserted into relation EMPLOYEE and a SAVEPOINT A is created.

```

mcarsept2023=# BEGIN;
BEGIN
mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*= (14729, 'VINI','F',101, 'XXXXX0052R');
INSERT 0 1
mcarsept2023=# SAVEPOINT A;
SAVEPOINT

```

```

mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |    M   |    101 | XXXXX0051R
 14729 |  VINI  |    F   |    101 | XXXXX0052R
(2 rows)

```

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT B is created.

```

mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*= (14730, 'ANI','F',102, 'XXXXX0053R');
INSERT 0 1
mcarsept2023=# SAVEPOINT B;
SAVEPOINT

```

```

mcarsept2023=# SELECT * FROM EMPLOYEE;
  eno |  ename | gender | dcode |   pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |    M   |    101 | XXXXX0051R
 14729 |  VINI  |    F   |    101 | XXXXX0052R
 14730 |  ANI   |    F   |    102 | XXXXX0053R
(3 rows)

```

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT C is created.

```

mcarsept2023=# INSERT INTO EMPLOYEE VALUES
mcarsept2023-*= (14731, 'MANJU','F',102, 'XXXXX0054R');
INSERT 0 1
mcarsept2023=# SAVEPOINT C;
SAVEPOINT

```

```
mcarsept2023=*# SELECT * FROM EMPLOYEE;
```

eno	ename	gender	dcode	pan
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R
14731	MANJU	F	102	XXXXX0054R

(4 rows)

There are four records in the EMPLOYEE relation. Now the TCL command ROLLBACK to B is executed.

```
mcarsept2023=*# ROLLBACK TO B;
```

```
ROLLBACK
```

```
mcarsept2023=*# SELECT * FROM EMPLOYEE;
```

eno	ename	gender	dcode	pan
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R

(3 rows)

It can be inferred that only three records are there in the EMPLOYEE relation after execution of ROLLBACK TO B.

```
mcarsept2023=*# COMMIT;
```

```
COMMIT
```

```
mcarsept2023=#
```

RENAME TABLE, TRUNCATE TABLE and DROP TABLE

```
mcarsept2023=# ALTER TABLE EMPLOYEE
```

```
mcarsept2023-# RENAME TO STAFF;
```

```
ALTER TABLE
```

```
mcarsept2023=# SELECT * FROM STAFF;
  eno |  ename | gender | dcode |      pan
-----+-----+-----+-----+-----
 14728 |  JOSE  |    M   |   101 | XXXXX0051R
 14729 |  VINI  |    F   |   101 | XXXXX0052R
 14730 |  ANI   |    F   |   102 | XXXXX0053R
(3 rows)
```

It can be inferred that relation EMPLOYEE is renamed to relation STAFF.

```
mcarsept2023=# TRUNCATE TABLE STAFF;
TRUNCATE TABLE
mcarsept2023=# SELECT * FROM STAFF;
  eno |  ename | gender | dcode |      pan
-----+-----+-----+-----+-----
(0 rows)
```

It can be inferred that after the relation STAFF is truncated there are no records in the STAFF relation.

Now, an attempt is done to drop the DEPARTMENT relation.

```
mcarsept2023=# DROP TABLE DEPARTMENT;
ERROR:  cannot drop table department because other objects depend on it
DETAIL:  constraint staff_dcode_fkey on table staff depends on table department
HINT:   Use DROP ... CASCADE to drop the dependent objects too.
```

It can be inferred, DCODE is a foreign key of STAFF relation referencing DCODE of DEPARTMENT relation. So, the DEPARTMENT relation cannot be dropped. First the STAFF relation has to be dropped followed by DEPARTMENT relation.

```
mcarsept2023=# DROP TABLE STAFF;
DROP TABLE
mcarsept2023=# DROP TABLE DEPARTMENT;
DROP TABLE
mcarsept2023=#
```

It can be inferred that relation STAFF and relation DEPARTMENT are dropped.

2.2.3. MYSQL Relational Database

A relation EMPLOYEE with two attributes namely ENO and ENAME is created using the following DDL command.

```
mysql> CREATE TABLE EMPLOYEE
-> (
-> ENO BIGINT,
-> ENAME VARCHAR(30)
-> );
Query OK, 0 rows affected (0.05 sec)
```

The datatype of attribute ENO is BIGINT and the data type of attribute ENAME is VARCHAR(30). This is an example for a poor design because no integrity constraints have been enforced.

```
mysql> DESCRIBE EMPLOYEE;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| ENO   | bigint        | YES  |     | NULL    |       |
| ENAME | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

INSERT INTO is an example for DML statement. Using the following INSERT INTO statements three records are inserted into the EMPLOYEE relation.

```
mysql> INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
Query OK, 1 row affected (0.01 sec)
```

SELECT statement in SQL is an example for DQL.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+
| ENO   | ENAME |
+-----+-----+
| 14728 | JOSE  |
| 14728 | MARY  |
| NULL  | ANSU  |
+-----+-----+
3 rows in set (0.00 sec)
```

It can be inferred that the values of attribute ENO of EMPLOYEE relation is duplicated and there is also NULL value for attribute ENO.

Using the DDL command ALTER TABLE an attempt is done to enforce PRIMARY KEY constraint on EMPLOYEE relation.

```
mysql> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);
ERROR 1138 (22004): Invalid use of NULL value
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there is a NULL value for the attribute ENO in EMPLOYEE relation.

Using the DML DELETE statement, the tuple that has NULL value for the attribute ENO in the EMPLOYEE relation is deleted.

```
mysql> DELETE FROM EMPLOYEE WHERE ENO IS NULL;
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+
| ENO   | ENAME |
+-----+-----+
| 14728 | JOSE  |
| 14728 | MARY  |
+-----+-----+
2 rows in set (0.00 sec)
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
mysql> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);  
ERROR 1062 (23000): Duplicate entry '14728' for key 'employee.PRIMARY'
```

It can be inferred that the PRIMARY KEY constraint cannot be enforced on EMPLOYEE relation since there are duplicate values for the attribute ENO in EMPLOYEE relation.

Now, using the DML DELETE statement, the tuples with DUPLICATE VALUES for the attribute ENO in the EMPLOYEE relation are deleted.

```
mysql> DELETE FROM EMPLOYEE WHERE ENO=14728;  
Query OK, 2 rows affected (0.01 sec)
```

Now, it can be inferred that there are no tuples in the EMPLOYEE relation.

```
mysql> SELECT * FROM EMPLOYEE;  
Empty set (0.00 sec)
```

Now, using the DDL statement ALTER TABLE, an attempt is done to enforce PRIMARY KEY constraint on attribute ENO of EMPLOYEE relation.

```
mysql> ALTER TABLE EMPLOYEE ADD PRIMARY KEY(ENO);  
Query OK, 0 rows affected (0.09 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

It can be inferred that the PRIMARY KEY constraint is enforced on attribute ENO of EMPLOYEE relation.


```
mysql> DESCRIBE EMPLOYEE;
+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| ENO   | bigint        | NO   | PRI | NULL    |       |
| ENAME | varchar(30)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

Now, an attempt is done to insert DUPLICATE values and NULL values for the attribute ENO of EMPLOYEE relation.

```
mysql> INSERT INTO EMPLOYEE VALUES(14728, 'JOSE');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EMPLOYEE VALUES(14728, 'MARY');
ERROR 1062 (23000): Duplicate entry '14728' for key 'employee.PRIMARY'
mysql> INSERT INTO EMPLOYEE VALUES(NULL, 'ANSU');
ERROR 1048 (23000): Column 'ENO' cannot be null
```

It can be inferred, since ENO is the Primary Key of EMPLOYEE relation, neither a duplicate value can be assigned to the attribute ENO nor NULL value can be assigned to attribute ENO.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+
| ENO   | ENAME |
+-----+-----+
| 14728 | JOSE  |
+-----+-----+
1 row in set (0.00 sec)
```

An attribute GENDER with data type CHAR and a CHECK CONSTRAINT GENDER IN ('M','F') is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
mysql> ALTER TABLE EMPLOYEE ADD GENDER CHAR CHECK(GENDER IN('M','F'));
Query OK, 3 rows affected (0.09 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

It can be inferred an attribute GENDER is added to the EMPLOYEE relation.

```
mysql> DESCRIBE EMPLOYEE;
```

Field	Type	Null	Key	Default	Extra
ENO	bigint	YES		NULL	
ENAME	varchar(30)	YES		NULL	
GENDER	char(1)	YES		NULL	

3 rows in set (0.00 sec)

```
mysql> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	GENDER
14728	JOSE	NULL

1 row in set (0.00 sec)

Now, an attempt is done to update the value of attribute GENDER of the EMPLOYEE relation using the DML UPDATE statement.

```
mysql> UPDATE EMPLOYEE SET GENDER='A' WHERE ENO=14728;  
ERROR 3819 (HY000): Check constraint 'employee_chk_1' is violated.
```

It can be inferred that the updating the value of attribute GENDER of EMPLOYEE relation failed, since an attempt has been done to update value of GENDER with the value 'A'.

```
mysql> UPDATE EMPLOYEE SET GENDER='M' WHERE ENO=14728;  
Query OK, 1 row affected (0.01 sec)  
Rows matched: 1  Changed: 1  Warnings: 0
```

It can be inferred that only the value 'M' / 'F' can be assigned to attribute GENDER of the EMPLOYEE relation.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+
| ENO   | ENAME | GENDER |
+-----+-----+-----+
| 14728 | JOSE  | M      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

A relation DEPARTMENT with two attributes namely DNO and DNAME is created using the following DDL statement. The data type of attribute DCODE is SMALLINT and the data type of DNAME is VARCHAR (30).

```
mysql> CREATE TABLE DEPARTMENT
-> (
-> DCODE SMALLINT,
-> DNAME VARCHAR(32)
-> );
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> DESCRIBE DEPARTMENT;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| DCODE | smallint      | YES  |     | NULL    |       |
| DNAME | varchar(32)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

An attribute DCODE with data type SMALLINT is added to the EMPLOYEE relation using the DDL command ALTER TABLE.

```
mysql> ALTER TABLE EMPLOYEE ADD DCODE SMALLINT;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

It can be inferred an attribute DCODE is added to the EMPLOYEE relation.

```
mysql> DESCRIBE EMPLOYEE;
```

Field	Type	Null	Key	Default	Extra
ENO	bigint	NO	PRI	NULL	
ENAME	varchar(30)	YES		NULL	
GENDER	char(1)	YES		NULL	
DCODE	smallint	YES		NULL	

```
4 rows in set (0.01 sec)
```

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
mysql> ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
-> REFERENCES DEPARTMENT(DCODE);
ERROR 1822 (HY000): Failed to add the foreign key constraint.
Missing index for constraint 'employee_ibfk_1'
in the referenced table 'department'
```

It can be inferred that since the DEPARTMENT relation has no PRIMARY KEY Constraint, the FOREIGN KEY Constraint cannot be enforced.

```
mysql> ALTER TABLE DEPARTMENT ADD PRIMARY KEY(DCODE);
Query OK, 0 rows affected (0.06 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

It can be inferred that primary key constraint is enforced on the DEPARTMENT relation. The PRIMARY KEY of DEPARTMENT relation is DCODE.

```
mysql> DESCRIBE DEPARTMENT;
```

Field	Type	Null	Key	Default	Extra
DCODE	smallint	NO	PRI	NULL	
DNAME	varchar(32)	YES		NULL	

2 rows in set (0.01 sec)

Now, an attempt is done to add a FOREIGN Key constraint using the DDL command ALTER TABLE. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing attribute DCODE of DEPARTMENT relation.

```
mysql> ALTER TABLE EMPLOYEE ADD FOREIGN KEY(DCODE)
-> REFERENCES DEPARTMENT(DCODE);
Query OK, 1 row affected (0.11 sec)
Records: 1 Duplicates: 0 Warnings: 0
```

It can be inferred that foreign key constraint is enforced on the EMPLOYEE relation. The attribute DCODE of EMPLOYEE relation is a FOREIGN KEY referencing the PRIMARY KEY DCODE of the DEPARTMENT relation.

```
mysql> DESCRIBE EMPLOYEE;
```

Field	Type	Null	Key	Default	Extra
ENO	bigint	NO	PRI	NULL	
ENAME	varchar(30)	YES		NULL	
GENDER	char(1)	YES		NULL	
DCODE	smallint	YES	MUL	NULL	

4 rows in set (0.01 sec)

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE |
+-----+-----+-----+-----+
| 14728 | JOSE  | M      | NULL  |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
mysql> UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
ERROR 1452 (23000): Cannot add or update a child row:
a foreign key constraint fails (`mcarsept2023`.`employee`,
CONSTRAINT `employee_ibfk_1` FOREIGN KEY (`DCODE`)
REFERENCES `department` (`DCODE`))
```

It can be inferred that the UPDATE statement fails. This is because the value 101 is not present in the DEPARTMENT relation. Now, a record is inserted in the DEPARTMENT relation with the value 101 for DCODE.

```
mysql> INSERT INTO DEPARTMENT VALUES(101, 'COMPUTER SCIENCE AND ENGINEERING');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| DCODE | DNAME                                     |
+-----+-----+
| 101   | COMPUTER SCIENCE AND ENGINEERING |
+-----+-----+
1 row in set (0.00 sec)
```

Now, an attempt is done to update the value of DCODE of EMPLOYEE relation using the DML statement UPDATE.

```
mysql> UPDATE EMPLOYEE SET DCODE=101 WHERE ENO=14728;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

It can be inferred after inserting a record in the DEPARTMENT relation with the value 101 for DCODE, the value of attribute DCODE in the EMPLOYEE relation is updated.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE |
+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

An attribute PAN with data type VARCHAR (10) and an UNIQUE Constraint is added to EMPLOYEE relation using the DDL command ALTER TABLE.

```
mysql> ALTER TABLE EMPLOYEE ADD PAN VARCHAR(10) UNIQUE;
Query OK, 0 rows affected (0.14 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

It can be inferred an attribute PAN is added to the EMPLOYEE relation.

```
mysql> DESCRIBE EMPLOYEE;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| ENO   | bigint        | NO   | PRI | NULL    |       |
| ENAME | varchar(30)   | YES  |     | NULL    |       |
| GENDER | char(1)       | YES  |     | NULL    |       |
| DCODE | smallint      | YES  | MUL | NULL    |       |
| PAN   | varchar(10)   | YES  | UNI | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN   |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | NULL  |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Now, an attempt is done to update the value of PAN of EMPLOYEE relation pertaining to ENO: 14728 using the DML statement UPDATE.

```
mysql> UPDATE EMPLOYEE SET PAN='XXXXX0051R'
-> WHERE ENO=14728;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

It can be inferred attribute PAN of EMPLOYEE relation pertaining to ENO: 14728 is updated.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN       |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | XXXXX0051R |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command.

```
mysql> INSERT INTO EMPLOYEE
-> VALUES(14729, 'VINI', 'F', 101, 'XXXXX0051R');
ERROR 1062 (23000): Duplicate entry 'XXXXX0051R' for key 'employee.PAN'
```

It can be inferred that the INSERT INTO DML Command fails, since an attempt is done to insert a duplicate value for PAN.

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
mysql> INSERT INTO EMPLOYEE VALUES  
-> (14729, 'VINI', 'F', 101, NULL);  
Query OK, 1 row affected (0.01 sec)
```

It can be inferred that the INSERT INTO DML Command succeeds, with NULL value for the value of PAN.

```
mysql> SELECT * FROM EMPLOYEE;  
+-----+-----+-----+-----+-----+  
| ENO   | ENAME | GENDER | DCODE | PAN           |  
+-----+-----+-----+-----+-----+  
| 14728 | JOSE  | M      | 101   | XXXXX0051R   |  
| 14729 | VINI  | F      | 101   | NULL         |  
+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
mysql> ALTER TABLE EMPLOYEE MODIFY PAN VARCHAR(10) NOT NULL;  
ERROR 1138 (22004): Invalid use of NULL value
```

It can be inferred that the ALTER TABLE Command fails since there is a NULL value for PAN pertaining to ENO: 14729. Now, the record pertaining to ENO: 14729 is deleted using the following command.

```
mysql> DELETE FROM EMPLOYEE WHERE ENO=14729;  
Query OK, 1 row affected (0.01 sec)
```

It can be inferred that the record pertaining to ENO: 14729 is deleted.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN          |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | XXXXX0051R  |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Now, an attempt is done to add a NOT NULL constraint using the DDL command ALTER TABLE.

```
mysql> ALTER TABLE EMPLOYEE MODIFY PAN VARCHAR(10) NOT NULL;
Query OK, 0 rows affected (0.11 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

It can be inferred that the EMPLOYEE relation is altered.

```
mysql> DESCRIBE EMPLOYEE;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| ENO   | bigint       | NO   | PRI | NULL    |       |
| ENAME | varchar(30)  | YES  |     | NULL    |       |
| GENDER | char(1)      | YES  |     | NULL    |       |
| DCODE | smallint     | YES  | MUL | NULL    |       |
| PAN   | varchar(10)  | NO   | UNI | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

Now, an attempt is done to insert a record into the EMPLOYEE relation using the INSERT INTO DML Command with NULL value for PAN.

```
mysql> INSERT INTO EMPLOYEE VALUES
-> (14729, 'VINI','F',101, NULL);
ERROR 1048 (23000): Column 'PAN' cannot be null
```

It can be inferred that the INSERT INTO DML Command fails, since a NOT NULL Constraint has been enforced.

COMMIT, ROLLBACK AND SAVEPOINT

An additional record is inserted to the DEPARTMENT relation.

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO DEPARTMENT VALUES
-> (102, 'INFORMATION SCIENCE');
Query OK, 1 row affected (0.02 sec)
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| DCODE | DNAME                                     |
+-----+-----+
| 101   | COMPUTER SCIENCE AND ENGINEERING      |
| 102   | INFORMATION SCIENCE                   |
+-----+-----+
2 rows in set (0.00 sec)
```

Now, the COMMIT command is executed.

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Now, the ROLLBACK command is executed.

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM DEPARTMENT;
+-----+-----+
| DCODE | DNAME                                     |
+-----+-----+
| 101   | COMPUTER SCIENCE AND ENGINEERING |
| 102   | INFORMATION SCIENCE               |
+-----+-----+
2 rows in set (0.00 sec)
```

It can be inferred that COMMIT and ROLLBACK are irrevokable actions.
The EMPLOYEE relation has one record.

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN           |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | XXXXX0051R   |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

Now, three additional records are inserted into the EMPLOYEE relation.

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (14729, 'VINI','F',101, 'XXXXX0052R');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (14730, 'ANI','F',102, 'XXXXX0053R');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (14731, 'MANJU','F',102, 'XXXXX0054R');
Query OK, 1 row affected (0.00 sec)
```

```
mysql> BEGIN;
mysql> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	GENDER	DCODE	PAN
14728	JOSE	M	101	XXXXXX0051R
14729	VINI	F	101	XXXXXX0052R
14730	ANI	F	102	XXXXXX0053R
14731	MANJU	F	102	XXXXXX0054R

```
4 rows in set (0.00 sec)
```

Now, the ROLLBACK command is executed.

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	GENDER	DCODE	PAN
14728	JOSE	M	101	XXXXXX0051R

```
1 row in set (0.00 sec)
```

It can be inferred that after executing the ROLLBACK command the EMPLOYEE relation has only one record.

Now, a record is inserted into relation EMPLOYEE and a SAVEPOINT A is created.

```
mysql> INSERT INTO EMPLOYEE VALUES
-> (14729, 'VINI', 'F', 101, 'XXXXXX0052R');
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN           |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | XXXXX0051R   |
| 14729 | VINI  | F      | 101   | XXXXX0052R   |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT B is created.

```
mysql> INSERT INTO EMPLOYEE VALUES
-> (14730, 'ANI', 'F', 102, 'XXXXX0053R');
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+-----+
| ENO   | ENAME | GENDER | DCODE | PAN           |
+-----+-----+-----+-----+-----+
| 14728 | JOSE  | M      | 101   | XXXXX0051R   |
| 14729 | VINI  | F      | 101   | XXXXX0052R   |
| 14730 | ANI   | F      | 102   | XXXXX0053R   |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

Now, another record is inserted into relation EMPLOYEE and a SAVEPOINT C is created.

```
mysql> INSERT INTO EMPLOYEE VALUES
-> (14731, 'MANJU', 'F', 102, 'XXXXX0054R');
Query OK, 1 row affected (0.00 sec)
```

```
mysql> SAVEPOINT C;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	GENDER	DCODE	PAN
14728	JOSE	M	101	XXXXXX0051R
14729	VINI	F	101	XXXXXX0052R
14730	ANI	F	102	XXXXXX0053R
14731	MANJU	F	102	XXXXXX0054R

```
4 rows in set (0.00 sec)
```

There are four records in the EMPLOYEE relation. Now the TCL command ROLLBACK to B is executed.

```
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
```

ENO	ENAME	GENDER	DCODE	PAN
14728	JOSE	M	101	XXXXXX0051R
14729	VINI	F	101	XXXXXX0052R
14730	ANI	F	102	XXXXXX0053R

```
3 rows in set (0.00 sec)
```

It can be inferred that only three records are there in the EMPLOYEE relation after execution of ROLLBACK TO B.

```
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

RENAME TABLE, TRUNCATE TABLE and DROP TABLE

```
mysql> RENAME TABLE EMPLOYEE TO STAFF;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> SELECT * FROM STAFF;
```

ENO	ENAME	GENDER	DCODE	PAN
14728	JOSE	M	101	XXXXX0051R
14729	VINI	F	101	XXXXX0052R
14730	ANI	F	102	XXXXX0053R

```
3 rows in set (0.00 sec)
```

It can be inferred that relation EMPLOYEE is renamed to relation STAFF.

```
mysql> TRUNCATE TABLE STAFF;
Query OK, 0 rows affected (0.07 sec)

mysql> SELECT * FROM STAFF;
Empty set (0.01 sec)
```

It can be inferred that after the relation STAFF is truncated there are no records in the STAFF relation.

Now, an attempt is done to drop the DEPARTMENT relation.

```
mysql> DROP TABLE DEPARTMENT;
ERROR 3730 (HY000): Cannot drop table 'department'
referenced by a foreign key constraint 'staff_ibfk_1' on table 'staff'.
```

It can be inferred, DCODE is a foreign key of STAFF relation referencing DCODE of DEPARTMENT relation. So, the DEPARTMENT relation cannot be dropped. First the STAFF relation has to be dropped followed by DEPARTMENT relation.

```
mysql> DROP TABLE STAFF;
Query OK, 0 rows affected (0.04 sec)

mysql> DROP TABLE DEPARTMENT;
Query OK, 0 rows affected (0.03 sec)
```


It can be inferred that relation STAFF and relation DEPARTMENT are dropped.

Learning Outcomes

Learner would have gained knowledge on the following:

- Need for Structured Query Language
- Creating Relations and Enforcing Integrity Constraints Using Data Definition Language
- Data Manipulation Language and Data Query Language
- Data Control Language and Transaction Control Language

Short Questions

3. Outline the need for structured query language.
4. Outline commit and rollback with an example.

Long Question

2. Outline the integrity constraints with an example.

Multiple Choice Questions

4. Create Table is an example for _____.
 - v. Data Definition Language
 - vi. Data Manipulation Language
 - vii. Data Control Language**
 - viii. Transaction Control Language**

Answer: i.

5. Update Table is an example for _____.
 - i. Data Definition Language
 - ii. Data Manipulation Language**
 - iii. Data Control Language
 - iv. Transaction Control Language

Answer: ii.

6. Delete From Table is an example for _____.
 - i. Data Definition Language
 - ii. Data Manipulation Language**

- iii. Data Control Language
- iv. Transaction Control Language

Answer: ii.

References

[2] Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Seventh Edition, Pearson Education, 2016.